is that the part before the colon refers to the name of the named volume rather than a local path. Here (line 23) we're saying, "Mount the db_data named volume at /var/lib/postgresql/data"—the directory where the Postgres image stores its database files that we want to persist.[7]

OK, let's give this a whirl. We've changed our Compose definition for our database service, so that's the one we need to restart. However, again, because Compose reuses the same container for a service unless we tell it otherwise, we have to explicitly tell Compose to recreate the database container to pick up our new volume settings.

First stop the database service:

```
$ docker-compose stop database
```

Then let's explicitly remove its container:

```
$ docker-compose rm -f database
```

Compose would normally ask us to confirm before removing the container—the force (-f) option tells it to just go ahead and do it.

OK, it's time to bring our database back up:

```
$ docker-compose up -d database
Creating volume "myapp_db_data" with default driver
Creating myapp_database_1 ... done
```

Since our new volume is now mounted in the container, we've wiped any previous databases and data we had stored, so we'll need to recreate and migrate the databases.

Let's do this now:

```
$ docker-compose exec web bin/rails db:create db:migrate
Database 'myapp_development' already exists
Created database 'myapp_test'
== 20190115100954 CreateUsers: migrating ==================================
-- create_table(:users)
   -> 0.0127s
== 20190115100954 CreateUsers: migrated (0.0143s) =========================
```

OK, now let's make sure our app is still working. Visit http://localhost:3000/users in the browser, and make sure you see our User scaffold. Great—the volume seems to be working.

7. https://hub.docker.com/_/postgres/